

CAMPUS LOST & FOUND MANAGEMENT **SYSTEM**

PROJECT REPORT

Name: SARTHAK JINDAL

Registration no.: 25BCE10189

Course: JAVA Programming (CSE2006)

Slot: B22+B24

1. ABSTRACT

Losing personal belongings like student ID cards, notebooks, chargers, or wallets is an everyday occurrence across university campuses. Without a centralized tracking mechanism, students often have to rely on word of mouth or multiple scattered inquiries to trace missing items. The Campus Lost & Found Management System is a practical, console-based Java application developed to streamline and organize this process.

The software provides an intuitive menu-driven environment where students and staff can register their profiles, log lost possessions, record discovered items, search existing logs, mark items as claimed, and remove outdated entries. To ensure data is not lost when the program terminates, the system integrates file-handling routines that automatically save and reload all user profiles and item records.

Through this project, key core Java principles are put into practice, including object-oriented modeling with classes and constructors, encapsulation, dynamic record tracking via ArrayLists, modular method design, loop control, user input validation, exception handling, and persistent text-file I/O.

2. INTRODUCTION

In a bustling academic campus, students continuously move between lecture halls, laboratories, food courts, libraries, and hostels. In the process, valuable belongings frequently get left behind. In conventional campus setups, lost and found inquiries are managed manually using register books at security desks or department offices, which often proves tedious, error-prone, and slow when students need to locate their items.

To address this challenge, the Campus Lost & Found Management System offers a clean, console-driven management tool. By giving each registered user and reported item a distinctive record, the system makes it straightforward to record descriptions, specify where items were misplaced or retrieved, query the logbook by item name, and update ownership status once an item is safely returned.

3. OBJECTIVES

The key objectives of building this project are as follows:

- To build an interactive, menu-driven console tool tailored for everyday campus use.
- To establish a neat and consistent record-keeping structure for misplaced and recovered items.
- To apply core Object-Oriented Programming (OOP) concepts such as custom Classes, Objects, and Constructors in a real-world scenario.
- To ensure proper Encapsulation through private fields and getter/setter access methods.
- To manage changing lists of records flexibly using Java's ArrayList collection.
- To structure program functionality into modular, reusable methods for clean code maintenance.
- To safeguard the program against runtime crashes by capturing improper inputs with Exception Handling.
- To establish basic persistence by reading from and writing to structured text files.

4. PROJECT SCOPE

The scope of this project is tailored to internal campus operations such as university departments, academic buildings, or hostel administration. It focuses on offering a reliable text-based tool to log, query, and manage property records without requiring complicated database engines.

Feature	Description & Purpose
User Registration	Enables students or faculty to register basic contact info and receive an identifier.
Lost Item Reporting	Logs complete details of missing items along with approximate locations.
Found Item Reporting	Records items found around the campus and tags them as available for claim.
View Items	Outputs the complete list of active lost and found records to the console.
Search Item	Allows users to quickly search the database by typing an item's name.
Claim Item	Updates the status of a found item once verified and returned to its

	owner.
Remove Item	Deletes obsolete, claimed, or mistakenly logged records from the system.
File Storage	Saves all changes to local disk files so records remain available across sessions.

5. JAVA CONCEPTS USED

Java Concept	Role in Project Implementation
Classes and Objects	Defined CampusLostAndFound as the driver class and Item as the data model representing lost/found entities.
Constructors	Used parameterized constructors within the Item class to initialize newly logged items directly.
Encapsulation	Declared class variables private and exposed controlled access through public getters and setters.
ArrayList	Employed dynamic lists to maintain and manipulate collections of users and items in memory.
Methods	Organized functionality into distinct procedural routines for registering, searching, listing, claiming, and saving.
Loops & Conditions	Utilized while loops for menu navigation and if-else / switch structures for routing user commands.
Exception Handling	Implemented try-catch blocks to catch InputMismatchException and prevent crashes on invalid console input.
this Keyword	Used inside constructor bodies and mutators to differentiate instance attributes from parameter names.
final Keyword	Applied to constant values such as static file path locations to avoid unintended modification.
File Handling	Leveraged Scanner, File, and PrintWriter / FileWriter classes to load and persist records in plain text.

6. PROJECT STRUCTURE

The program is organized into a straightforward folder layout that isolates Java source files from saved flat-file data:

File / Directory	Description of Contents
CampusLostAndFound/	Root project directory containing all source code and storage folders.
src/	Source code folder housing all compiled and working Java files.
src/CampusLostAndFound.java	Main executable class responsible for console menus, user input, logic, and file I/O.
src/Item.java	Model class encapsulating item details, fields, constructors, and accessor methods.
data/	Dedicated storage folder used for holding runtime persistent data files.
data/users.txt	Text file containing line-delimited records of registered user profiles.

data/items.txt	Text file storing delimiter-separated records of all lost and found entries.
----------------	--

7. FILE HANDLING

To provide reliable data persistence without adding the complexity of an external database server, the application relies on lightweight, pipe-delimited text files stored within the local data folder.

User Data File (data/users.txt)

Registered user information is written sequentially with values separated by the '|' delimiter:

```
1|Sarthak|9876543210|sarthak@gmail.com
```

Item Data File (data/items.txt)

Item logs are similarly saved in plain text with item ID, item name, description, place found/lost, category, and status:

```
1001|Black Wallet|Black leather wallet with college ID|Hostel Block 4|Lost|Active
```

When the application starts, it verifies whether these files exist. If found, it reads each record line-by-line, parses the delimited fields, constructs the corresponding objects, and populates the in-memory ArrayLists. When users choose to save records or close the application, the updated lists are written back to disk.

8. ADVANTAGES

- Simple and intuitive text menu, making it easy for any student or staff member to operate immediately.
- Brings centralized structure and quick retrieval to campus lost-and-found tracking.
- Assigns individual, unique tracking IDs to eliminate confusion across similar items.
- Operates completely stand-alone using lightweight file handling, with zero external database dependencies.
- Demonstrates clean modular programming and effective practical use of core Java fundamentals.
- Easy to read, test, and expand with further modules in future iterations.

9. LIMITATIONS

- Currently restricted to a console/terminal interface without graphical visual components.
- Uses flat text files, which are ideal for basic usage but less scalable than relational databases (e.g., MySQL) for massive volumes of data.
- Does not support simultaneous remote or multi-client network access.
- Does not allow uploading or displaying photographic evidence of lost or found belongings.

- Matching between lost items and found items is done manually by title lookup rather than through an automated keyword-matching algorithm.

10. CONCLUSION

The Campus Lost & Found Management System presents a practical, cohesive application of core Java concepts to solve a common campus challenge. By combining basic object-oriented principles, dynamic collection handling, structured method division, and file I/O, the program provides a functional, dependable console tool for tracking misplaced student belongings.

The application serves as a strong demonstration of foundational Java programming, validating how theoretical constructs like encapsulation, exception handling, and input processing can be directly applied to build a useable, real-world campus management utility.